

When Should I Use a Struct Instead of a Class in C#?

In C#, choosing between **struct** and **class** depends mainly on the nature of the data you are modeling and how it will be used in your application. Below are clear guidelines to help you decide.

When to Use a Struct

- When the type represents a small amount of data (usually less than 16–32 bytes).
- When the type represents a value rather than an entity with identity (e.g., Point, DateTime).
- When immutability is preferred (values should not change after creation).
- When you do not need inheritance (structs do not support inheritance).
- When performance is critical and many instances are created (to reduce garbage collection overhead).

When Not to Use a Struct

- When the object is large or contains many fields.
- When the object is frequently modified (mutable).
- When you need inheritance or polymorphism.
- When null values are required.
- When passing the object frequently between methods (copying cost can increase).

Key Differences Between Struct and Class

Struct: Value type, stored on the stack, copied by value, cannot inherit, faster for small data.

Class: Reference type, stored on the heap, copied by reference, supports inheritance, more flexible.

Golden Rule

Use **class** by default. Choose **struct** only when the type is small, represents a value, is immutable, and performance is important.